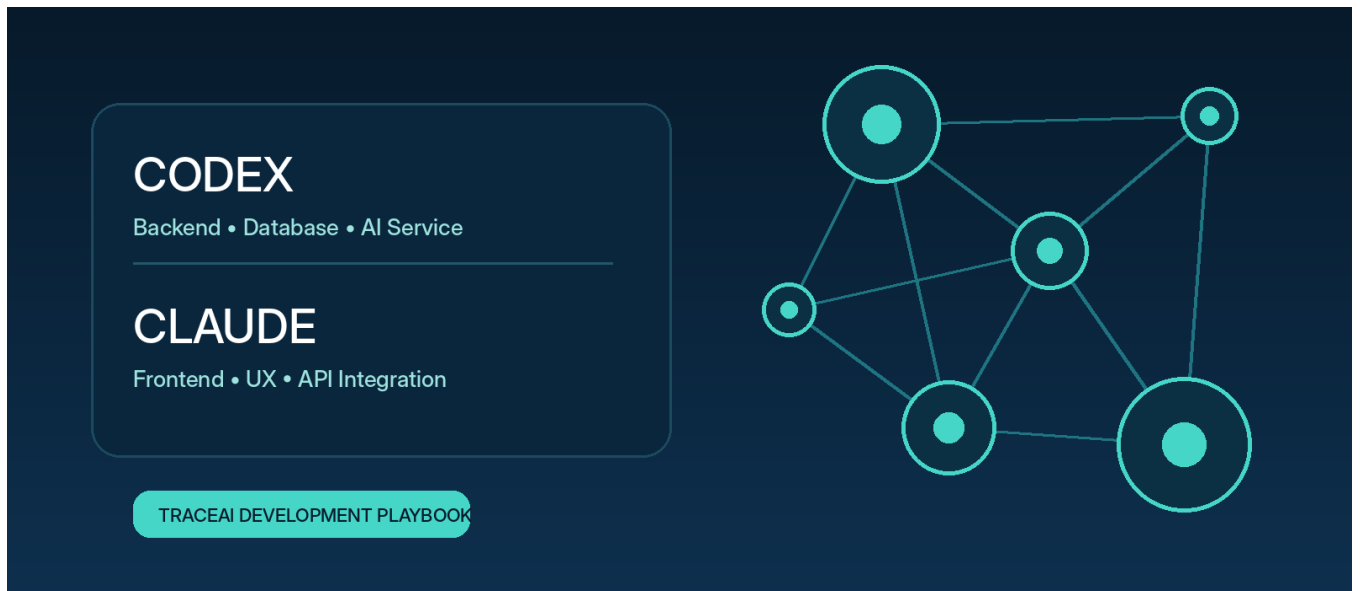




AI-Assisted Development Workflow

Using Codex for the backend, database and AI service - and Claude for the frontend.

PURPOSE

A practical, feature-by-feature playbook for building TraceAI with clear tool ownership, a shared API contract, safe Git integration and repeatable quality checks.

Prepared for Adam • June 2026

OVERVIEW

How to use this playbook

The document is organised around one principle: Codex defines how the system works; Claude defines how the user interacts with it; you verify every integration before moving forward.

01**Roles and ownership**

Clear responsibilities and protected folders.

02**Repository and architecture**

One repository, three services and shared documentation.

03**The delivery loop**

Build one complete feature at a time.

04**Git and integration**

Branch safely and prevent conflicting changes.

05**Prompts and instructions**

Reusable starting prompts for Codex and Claude.

06**Roadmap and quality gates**

A 12-stage implementation plan and final checklist.

CORE RULE

Do not ask either AI tool to build the complete application in one request. Create one backend capability, connect one frontend flow, test the integration and only then continue.

01 / ROLES

A deliberate division of work

The split is effective only when each tool has a clearly defined area and both tools use the same API contract.

Codex SYSTEM ENGINE - Node.js + TypeScript backend - PostgreSQL and migrations - Python FastAPI AI service - OpenAPI and API documentation - Tests, Docker and CI/CD	Claude PRODUCT INTERFACE - React + TypeScript frontend - Page layouts and reusable components - Responsive styling and accessibility - Form validation and user feedback - API client and UI integration	You OWNER & REVIEWER - Approve architecture decisions - Run services and tests - Review every generated change - Resolve integration issues - Commit, document and demonstrate
--	--	--

Folder ownership

Protecting folder boundaries prevents one tool from rewriting the other tool's work. Use these defaults unless a task explicitly requires cross-service changes.

Path	Owner	Purpose
frontend/	Claude	React application and frontend tests
backend/	Codex	Node.js API and backend tests
ai-service/	Codex	Python AI pipeline and tests
database/	Codex	Migrations, seeds and schema
docs/openapi.yaml	Codex	Source of truth for API request and response contracts
Final integration	You	Manual verification across all services

BOUNDARY RULE

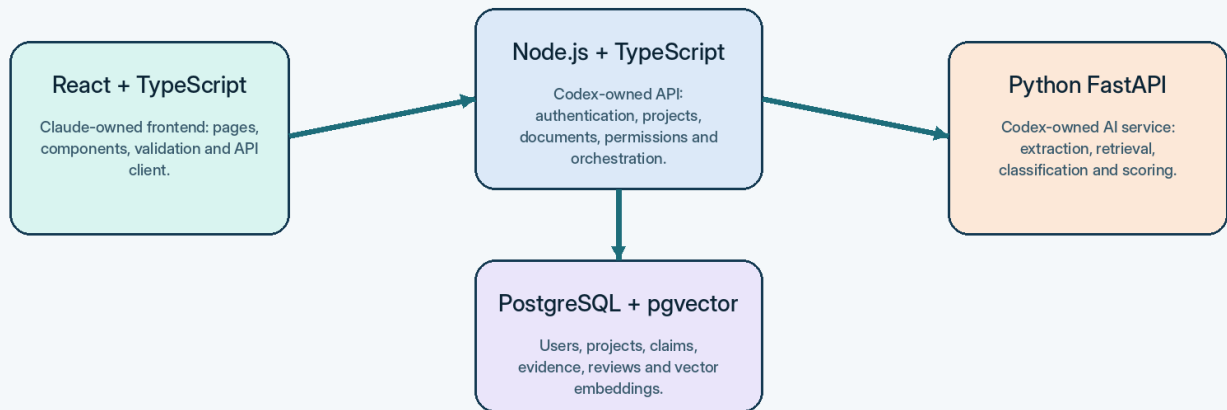
Codex should not modify frontend/ unless you explicitly ask it to fix a cross-service integration issue. Claude should not edit backend/, ai-service/, database/ or the OpenAPI contract.

02 / ARCHITECTURE

One repository, shared contracts

Keep the entire application in a single repository so each tool can read the same documentation, but maintain strong service boundaries.

TraceAI Service Architecture



Recommended repository structure

REPOSITORY LAYOUT

```

traceai/
├── frontend/           # Claude owns this
├── backend/            # Codex owns this
├── ai-service/         # Codex owns this
├── database/           # migrations, seeds, schema
├── docs/
│   ├── PROJECT_SPEC.md
│   ├── API_CONTRACT.md
│   ├── DATABASE_SCHEMA.md
│   └── openapi.yaml
├── docker-compose.yml
├── .env.example
├── .gitignore
├── AGENTS.md
└── README.md
  
```

Why the contract comes first

The frontend must know the exact endpoint, request body, response body, status codes and error format before integration begins. Codex creates and maintains that contract; Claude consumes it without inventing alternative endpoints.

SMALL CONTRACT EXAMPLE

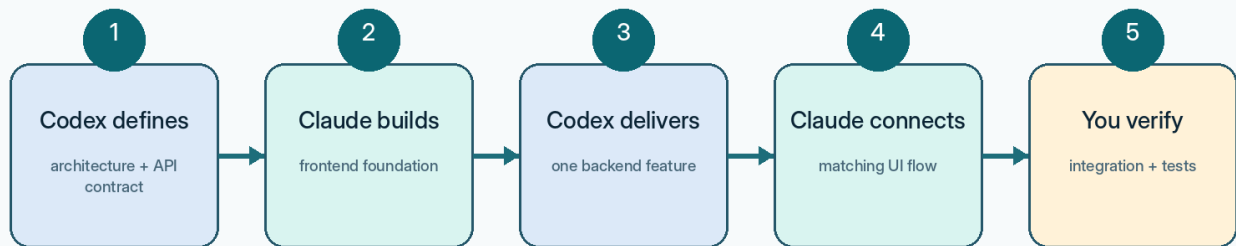
```

POST /api/auth/login
Request: { "email": "adam@example.com", "password": "..." }
Response: { "user": { ... }, "accessToken": "..." }
  
```

03 / DELIVERY LOOP

Build vertical slices, not isolated layers

A vertical slice is one complete feature that moves from the database to the API and then to a usable screen. It is easier to test, review and demonstrate.



Start with the foundation

1. Codex creates the repository structure, database connection, backend service, AI service, Docker setup and initial OpenAPI contract.
2. Claude creates the frontend shell, routing, reusable design system, API client structure and empty pages based on the contract.
3. You confirm all services run and the frontend builds before feature work begins.

Then repeat the same loop for every feature

4. Codex implements the backend capability, validation, database migration and automated tests.
5. Codex updates openapi.yaml and the API contract documentation.
6. You run the backend tests and manually inspect the endpoint.
7. Claude builds the matching UI using the documented request and response.
8. You test the complete user journey and only then merge the work.

DEFINITION OF COMPLETE

A feature is not complete because the code exists. It is complete when the database, API, user interface, validation, errors, tests and documentation all work together.

04 / FEATURES

Recommended implementation order

The sequence below reduces dependency problems and gives you a working product at the end of every stage.

Stage	Deliverable	Acceptance journey
1. Authentication	Users, password hashing, JWT, protected routes and logout.	Register → login → open dashboard → logout
2. Projects	Create, list and open verification projects with ownership checks.	Create a project and see it in the dashboard
3. Document upload	PDF upload, file validation, storage, extraction and processing status.	Upload a PDF and track its status
4. AI response analysis	Prompt + answer submission, claim extraction and analysis records.	Submit an answer and view extracted claims
5. Evidence retrieval	Chunking, embeddings, pgvector search and evidence references.	Open a claim and inspect the strongest evidence
6. Claim classification	Supported, partially supported, contradicted and unverifiable results.	Review a scored claim with confidence
7. Human review	Accept, correct or mark an assessment as needing more evidence.	Save a human decision and audit trail
8. Dashboard	Reliability metrics, topic/model comparisons and export.	Filter charts and export anonymised data

Example: authentication responsibilities

Codex delivers BACKEND • users migration and constraints • register/login endpoints • password hashing and JWT • request validation and error responses • endpoint and integration tests	Claude delivers FRONTEND • login and registration pages • authentication context • protected routes • field and server-error feedback • logout and session handling
---	--

05 / GIT

Integrate without losing work

Use small branches and merge only after the related service passes its own tests and the full user journey works.

BRANCH PATTERN

```
main
develop
backend/phase-1-authentication
frontend/phase-1-authentication
backend/document-processing
frontend/document-upload
```

Safe merge sequence

9. Codex completes a backend task on a backend branch.
10. You review the changed files, run tests and manually verify the API.
11. Merge the backend branch into develop.
12. Give Claude the updated develop branch and the matching frontend task.
13. Claude completes the UI integration on a frontend branch.
14. You run the complete feature locally and merge into develop.
15. Merge develop into main only at stable milestones.

Commit examples

MEANINGFUL COMMIT HISTORY

```
feat(auth): add JWT registration and login
feat(documents): implement validated PDF upload
feat(ai): add claim extraction endpoint
feat(retrieval): store and query vector embeddings
test(analysis): add API integration coverage
docs(setup): document Docker development workflow
```

AVOID

Do not paste unfinished AI-generated code directly into main. Do not allow both tools to perform large changes in the same files at the same time.

06 / REPOSITORY GUIDANCE

Make the rules persistent

Store architecture, commands, restrictions and completion requirements inside the repository so Codex receives consistent instructions on every task.

SUGGESTED AGENTS.MD

```
# TraceAI Development Instructions

## Ownership
- /backend, /ai-service, /database: Codex-owned.
- /frontend: do not modify unless explicitly requested.
- /docs/openapi.yaml: source of truth for the API contract.

## Backend rules
- TypeScript strict mode.
- Controllers, services, repositories, routes and middleware.
- Validate all input and never expose secrets.
- Add migrations and automated tests.
- Update OpenAPI whenever an endpoint changes.

## Completion requirements
1. Run linting and type checking.
2. Run automated tests.
3. Update documentation.
4. Summarise changed files.
5. Explain how the feature can be tested.
```

Environment variables

SAFE TEMPLATES ONLY

```
# backend/.env.example
DATABASE_URL=
JWT_SECRET=
AI_SERVICE_URL=http://ai-service:8000
PORT=5000

# frontend/.env.example
VITE_API_BASE_URL=http://localhost:5000/api

# ai-service/.env.example
DATABASE_URL=
MODEL_NAME=
EMBEDDING_MODEL=
```

SECURITY REQUIREMENT

Commit .env.example, but never commit .env, real passwords, API keys, JWT secrets, private user uploads or production database exports.

07 / PROMPT PACK

First Codex prompt

Use this prompt after creating the empty repository and placing the TraceAI proposal inside docs/.

COPY-READY PROMPT

```
You are responsible for the backend, PostgreSQL database, and Python AI service of the TraceAI project.

First, read the project proposal and inspect the repository. Do not implement the complete application yet.

Prepare Phase 1: architecture and project foundation.

Create:
1. A Node.js, Express, and TypeScript backend structure.
2. A Python FastAPI AI-service structure.
3. A PostgreSQL database structure.
4. Docker Compose for PostgreSQL, backend, and AI service.
5. A secure .env.example without real secrets.
6. Database migration and seed folders.
7. An initial OpenAPI specification.
8. PROJECT_SPEC.md, API_CONTRACT.md, and DATABASE_SCHEMA.md.
9. Health-check endpoints for both backend services.
10. Basic linting, formatting, and testing configuration.

The frontend will be created separately by Claude. Do not create or modify frontend application code.

Ownership boundaries:
- backend/, ai-service/, database/ and backend documentation: you may modify.
- frontend/: do not modify.

Before coding, present your architecture plan.

The task is complete when:
- All services start using Docker Compose.
- Both health endpoints work.
- PostgreSQL connects successfully.
- Tests and type checking pass.
- Setup instructions are included in the README.
```

08 / PROMPT PACK

First Claude prompt

Run this after Codex has committed the foundation and the OpenAPI contract is available.

COPY-READY PROMPT

```
You are responsible only for the frontend of the TraceAI project.

Read:
- The TraceAI project proposal
- docs/PROJECT_SPEC.md
- docs/API_CONTRACT.md
- docs/openapi.yaml
- README.md

The backend, database, and AI service are being developed separately by Codex.

Do not modify:
- backend/
- ai-service/
- database/
- docker-compose.yml
- docs/openapi.yaml

Create the frontend foundation using React, TypeScript, Vite, React Router, Axios and a structured reusable styling system.

Create:
1. Application layout and navigation.
2. Landing, login and registration pages.
3. An empty user dashboard.
4. Reusable buttons, forms, cards, modals, loaders and alerts.
5. API client structure and authentication context foundation.
6. Responsive desktop, tablet and mobile layouts.

Use mock data only where the backend endpoint is not implemented. Keep mocks separated from production API services. Do not invent endpoints; follow docs/openapi.yaml exactly.

Before coding, present the proposed frontend architecture and page design.

The task is complete when:
- The frontend builds without errors.
- TypeScript checking passes.
- Routes work.
- Pages are responsive.
- Components are reusable.
- No backend files have been changed.
```

09 / INTEGRATION

Mocks without future cleanup problems

Claude can create pages before every backend endpoint is ready, but mock data must be isolated so it can be replaced by one service change rather than edits across many components.

FRONTEND SEPARATION

```
frontend/src/
├── api/
│   ├── client.ts
│   └── analysisApi.ts
├── mocks/
│   ├── analysis.mock.ts
│   └── project.mock.ts
└── features/
    └── analysis/
```

MOCK-TO-REAL TRANSITION

```
// Development mock
const mockClaims = [
  {
    id: 1,
    text: 'The World Wide Web was invented by Bill Gates.',
    classification: 'contradicted',
    confidence: 0.96
  }
];

// Replace the service, not every component
getMockAnalysis() -> analysisApi.getAnalysis(id)
```

Standard error response

Define one reusable API error format so Claude can implement consistent form feedback, alerts and retry states.

RECOMMENDED CONTRACT

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "The request contains invalid fields.",
    "details": [
      { "field": "email", "message": "Enter a valid email." }
    ]
  }
}
```

INTEGRATION DISCIPLINE

Claude must use the documented endpoint and data type. If the contract is wrong, update it through Codex first rather than hiding the mismatch inside frontend code.

10 / ROADMAP

The complete 12-stage order

This sequence balances visible progress with technical foundations. Each stage should end with a small demonstration and a clean commit history.

Stage	Owner	Outcome
01	Codex	Architecture, API contract and Docker foundation
02	Claude	Frontend foundation and design system
03	Codex	Authentication backend
04	Claude	Authentication frontend
05	Codex	Projects and document backend
06	Claude	Projects and upload screens
07	Codex	Python claim-extraction pipeline
08	Claude	Analysis input and result interface
09	Codex	Evidence retrieval and dashboard endpoints
10	Claude	Evidence viewer, filters and charts
11	Codex	Testing, security, performance and deployment
12	Claude + You	Responsive polish, accessibility and final integration

Milestone evidence

- A short README update describing what changed and how to run it.
- Passing lint, type checking and automated tests for the service that changed.
- A screenshot or short screen recording of the completed user journey.
- An updated API contract whenever requests or responses changed.
- A meaningful commit history rather than one final bulk commit.

11 / QUALITY GATE

Before calling a phase complete

Use this checklist after every vertical slice. It keeps the project portfolio-ready throughout development instead of postponing quality until the end.

Architecture	- Folder ownership respected - No duplicate business logic across services - OpenAPI contract updated
Backend	- Input validation and permissions implemented - Database migration included - Tests pass and errors are consistent
Frontend	- Loading, empty, success and error states exist - Responsive layout checked - No endpoint invented outside the contract
Security	- No real secrets committed - Uploads and file sizes validated - Authentication and ownership checks enforced
Delivery	- Feature runs locally from documented steps - User journey manually tested - Changes committed on the correct branch
FINAL PRINCIPLE Codex defines how the data works. Claude defines how the user interacts with it. Both follow the same API contract, and you approve every merge.	

Your first action

16. Create the TraceAI repository with the recommended folders.
17. Place the project proposal inside docs/.
18. Add the AGENTS.md rules and .gitignore before generating code.
19. Run the first Codex prompt and review the proposed architecture before accepting implementation.
20. After the foundation passes its checks, run the first Claude prompt.